MikoshiLang

Reference Manual & User Guide

Mikoshi Ltd

Version 3.8.0 — November 1, 2026

Abstract

MikoshiLang is an open-source, Wolfram-style symbolic computation language for Python, built on SymPy and NumPy. It implements **4,402 of the 6,716** documented Wolfram Language symbols with reviewed, value-tested semantics (92.9% of the 4,739 judged feasible without curated data, notebooks or cloud services; see `docs/wolfram_coverage_roadmap.md`). Its rule is: a correct result, a clear error, or left unevaluated, never silently wrong. Version 3.8 adds three pillars: `Verify[]`, which re-checks results with an independent method and labels them Certified, Checked, Unchecked or Failed; two-way Python interop; and opt-in types read from Python annotations. MikoshiLang is alpha software, not a Mathematica replacement or a proof assistant.

GitHub: https://github.com/DarrenEdwards111/MikoshiLang
PyPI: `pip install mikoshilang`

Status of this manual (3.8)

Releases up to 3.7.1 advertised "6,324 functions". An audit found that figure was not supported: many entries were generated stubs and some gave wrong answers. Those modules are quarantined in 3.8 and are not registered by the evaluator. **Where any later section of this manual conflicts with the README, the README and the files in `docs/` take precedence.** In particular, the natural-language mode, the hosted web console and REST API, and the domain packs described below as "chemistry", "physics", "biology", "machine learning" and engineering functions belong to the pre-3.8 releases; the hosted services are offline and many of those functions are quarantined. Current, authoritative references:

- README: https://github.com/DarrenEdwards111/MikoshiLang#readme
- Coverage roadmap: `docs/wolfram_coverage_roadmap.md`
- Verification: `docs/verification.md`; types: `docs/types.md`; Python interop: `docs/python_interop.md`
- Runnable examples: `examples/`

Contents

1 Introduction **6**
1.1 Key Features . . . 6
1.2 Installation . . . 6
1.2.1 Linux / macOS . . . 6
1.2.2 Windows (Anaconda — Recommended) . . . 7
1.2.3 Windows (WSL — Alternative) . . . 7
1.2.4 macOS (Apple Silicon / Intel) . . . 7
1.2.5 Optional Dependencies . . . 7
1.3 Quick Start . . . 8

2 Knowledge Layer (v3.5.0) **8**
2.1 Architecture . . . 8
2.2 Core Knowledge Functions . . . 8
2.2.1 EntitySearch . . . 8
2.2.2 EntityValue . . . 9
2.2.3 EntityRelationships . . . 9
2.2.4 WikipediaText . . . 9
2.3 Domain Packs . . . 9
2.3.1 PubChem (Chemistry) . . . 9
2.3.2 Crossref (Scholarly Papers) . . . 9
2.3.3 OpenAlex (Scholarly Graph) . . . 10
2.3.4 GeoNames (Geographic Data) . . . 10
2.3.5 World Bank (Economic Indicators) . . . 10
2.3.6 AlphaFold (Protein Structures) . . . 10
2.3.7 JHTDB (Turbulence Simulations) . . . 10
2.4 Caching & Provenance . . . 11
2.5 License Compliance . . . 11

3 Syntax **11**
3.1 Basic Expressions . . . 11
3.2 Function Calls . . . 12
3.3 Pattern Matching . . . 12

4 Core Functions **12**
4.1 Calculus . . . 12
4.1.1 Differentiation . . . 12
4.1.2 Integration . . . 12
4.1.3 Limits . . . 12
4.2 Algebra . . . 12
4.2.1 Solving Equations . . . 12
4.2.2 Simplification . . . 13
4.3 Linear Algebra . . . 13
4.4 Number Theory . . . 13
4.5 Statistics . . . 13

5 Visualization **13**
5.1 2D Plotting . . . 13
5.2 3D Plotting . . . 14
5.3 Interactive Plots . . . 14

6 Chemistry **14**
6.1 Equation Balancing . . . 14
6.2 Molecular Mass . . . 14
6.3 Element Data . . . 14

7 Unit Conversion **14**

8 Natural Language Mode **15**
8.1 How It Works . . . 15
8.2 Examples . . . 15
8.3 Web Console . . . 15

9 API Usage **15**
9.1 Python Library . . . 15
9.2 REST API . . . 16
9.3 Jupyter Kernel . . . 16
9.3.1 Installation . . . 16
9.3.2 Platform Support . . . 16
9.3.3 Usage . . . 16
9.3.4 Troubleshooting . . . 17

10 Function Reference **17**
10.1 Version History . . . 17
10.2 Extended Function Sets (v2.0+) . . . 18
10.3 Domain Coverage . . . 18
10.4 Calculus (67 functions) . . . 19
10.5 Algebra (89 functions) . . . 19
10.6 Linear Algebra (76 functions) . . . 19
10.7 Number Theory (94 functions) . . . 19
10.8 Statistics (112 functions) . . . 19
10.9 Trigonometry (48 functions) . . . 19
10.10Special Functions (97 functions) . . . 19
10.11Discrete Math (68 functions) . . . 20
10.12Logic & Sets (34 functions) . . . 20
10.13String Operations (52 functions) . . . 20
10.14List Manipulation (89 functions) . . . 20
10.15Graph Theory (64 functions) . . . 20
10.16Chemistry (118 elements + functions) . . . 20
10.17Units (50+ units) . . . 20
10.18Visualization (9 functions) . . . 20
10.19Numerical Methods (43 functions) . . . 21
10.20Optimization (37 functions) . . . 21
10.21Control Theory (28 functions) . . . 21
10.22Signal Processing (46 functions) . . . 21
10.23Time & Date (24 functions) . . . 21
10.24Random & Probability (38 functions) . . . 21
10.25Financial Math (21 functions) . . . 21

11 New Domain Examples (v2.0) **21**
11.1 Vector Calculus . . . 21
11.2 Machine Learning . . . 22
11.3 Physics . . . 22
11.4 Biology & Genetics . . . 22
11.5 Cryptography . . . 23
11.6 Numerical Methods . . . 23

12 Advanced Topics **23**
12.1 Custom Rules . . . 23
12.2 Pattern Matching . . . 24
12.3 Performance Tips . . . 24

13 Troubleshooting **24**
13.1 Common Errors . . . 24
13.2 Getting Help . . . 24

14 Contributing **24**

15 License **25**

16 Acknowledgments **25**

1 Introduction

MikoshiLang is a symbolic computation language designed for Python, providing Wolfram-style syntax with deterministic evaluation. Built on SymPy, it extends Python's mathematical capabilities with pattern matching, rule-based transformations, and natural language processing.

1.1 Key Features

- **4,402 documented Wolfram symbols** implemented with reviewed, value-tested semantics (see the coverage roadmap)
- **Verification** — `Verify[expr]` returns Certified, Checked, Unchecked or Failed, using a method independent of the one that produced the result
- **Python interop** — results as Python data, typed Python functions as language heads, Python access off unless allow-listed
- **Opt-in types** — signatures from Python annotations; `Session(check_types=True)`
- **Pattern matching & rules** — symbolic manipulation with custom transformations
- **Jupyter kernel** — interactive notebooks with rich output
- **Visualization** — 2D/3D plotting with matplotlib & plotly
- **Chemistry** — equation balancing, molecular mass, 118 elements
- **Physics** — mechanics, thermodynamics, E&M, quantum, relativity
- **Machine learning** — regression, neural nets, loss functions, optimizers
- **Biology** — genetics, DNA analysis, population dynamics
- **Cryptography** — hashing, modular arithmetic, number theory
- **Unit conversion** — 50+ physical units with dimensional analysis

1.2 Installation

1.2.1 Linux / macOS

```
# Create virtual environment (recommended)
python3 -m venv ~/mikoshi-env
source ~/mikoshi-env/bin/activate

# Install MikoshiLang
pip install mikoshilang

# With Jupyter support
pip install mikoshilang[jupyter]

# With all features (Jupyter + visualization + signal processing)
pip install mikoshilang[all]
```

Note: On Ubuntu 24.04+ the virtual environment is required due to PEP 668 (externally-managed-environment).

1.2.2 Windows (Anaconda — Recommended)

```
# Create new conda environment
conda create -n mikoshilang python=3.9
conda activate mikoshilang

# Install MikoshiLang with Jupyter support
pip install mikoshilang[jupyter]

# All features
pip install mikoshilang[all]
```

Supported on all platforms: Windows, macOS, Linux, WSL. The Jupyter kernel is fully compatible with Anaconda (fixed in v3.6.9 with `nest_asyncio` integration).

1.2.3 Windows (WSL — Alternative)

```
# Install WSL (if not already installed)
# In PowerShell: wsl --install

# Open WSL terminal (Ubuntu)
# Then follow Linux installation steps:
python3 -m venv ~/mikoshi-env
source ~/mikoshi-env/bin/activate
pip install mikoshilang[jupyter]

# Start Jupyter
jupyter notebook

# Access from Windows browser at http://localhost:8888
```

1.2.4 macOS (Apple Silicon / Intel)

```
# Create virtual environment
python3 -m venv ~/mikoshi-env
source ~/mikoshi-env/bin/activate

# Install MikoshiLang
pip install mikoshilang[jupyter]

# For M1/M2/M3 Macs: all dependencies are ARM64-compatible
```

1.2.5 Optional Dependencies

```
# Jupyter notebook support
pip install mikoshilang[jupyter]

# Visualization (matplotlib + plotly)
pip install mikoshilang[visualization]

# Signal processing (scipy)
pip install mikoshilang[signal]

# Everything
pip install mikoshilang[all]
```

1.3 Quick Start

```python
from mikoshilang import parse_and_eval

# Basic arithmetic
parse_and_eval("2 + 2")
# => 4

# Symbolic calculus
parse_and_eval("D[x^3 + 2*x, x]")
# => 3*x^2 + 2

# Equation solving
parse_and_eval("Solve[x^2 - 4 == 0, x]")
# => {-2, 2}

# Plotting
parse_and_eval("Plot[sin(x), {x, 0, 6.28}]")
# => <base64 PNG image>
```

2 Knowledge Layer (v3.5.0)

MikoshiLang v3.5.0 introduces a complete knowledge layer with deterministic fact retrieval from multiple sources. Unlike LLMs that answer from memory, MikoshiLang queries structured databases and returns provenance-tracked results.

2.1 Architecture

The knowledge layer implements 8 core components:

1. **Entity Graph** — Wikidata (QIDs, 50 canonical properties)
2. **Text Layer** — Wikipedia (summaries, citations)
3. **5 Core Tools** — Search, facts, relationships, text, time-travel
4. **LLM Interpreter** — Natural language → structured queries
5. **Persistent Cache** — SQLite with TTL (7d Wikidata, 24h Wikipedia)
6. **Canonical Properties** — Stable schema (BirthDate, Occupation, etc.)
7. **Domain Packs** — PubChem, Crossref, OpenAlex, GeoNames, World Bank, AlphaFold, JHTDB
8. **License Compliance** — Full attribution (CC0, CC BY-SA, Public Domain)

2.2 Core Knowledge Functions

2.2.1 EntitySearch

Search Wikidata entities:

```
EntitySearch["Douglas Adams"]
# => [{"id": "Q42", "label": "Douglas Adams",
#      "description": "British science fiction writer...",
#      "license": "CC0"}]

EntitySearch["Moon", "Film", 10]  # Type hint, limit 10
```

2.2.2 EntityValue

Get entity properties:

```
EntityValue["Q42", "BirthDate"]
# => {"value": "1952-03-11", "license": "CC0",
#     "source": "https://www.wikidata.org/wiki/Q42"}

# Time-travel queries
EntityValue["Q42", "Occupation", as_of="2015-01-01"]
```

Canonical Properties (50): BirthDate, DeathDate, Occupation, Country, Population, ChemicalFormula, AtomicNumber, ISBN, DOI, and more.

2.2.3 EntityRelationships

SPARQL-powered graph traversal:

```
EntityRelationships["Q42", "Occupation", depth=1, limit=10]
# => {"related": [{"id": "Q36180", "label": "writer"}, ...],
#     "license": "CC0"}
```

2.2.4 WikipediaText

Get Wikipedia summaries with attribution:

```
WikipediaText["Douglas Adams", sentences=3]
# => {"text": "Douglas Noel Adams was...",
#     "license": "CC BY-SA 3.0",
#     "attribution": "Source: Wikipedia contributors"}
```

2.3 Domain Packs

MikoshiLang includes 7 domain-specific knowledge sources:

2.3.1 PubChem (Chemistry)

```
PackSearch["pubchem", "caffeine"]
# => [{"id": "CID2519", "source": "PubChem",
#      "license": "Public Domain"}]

PackValue["pubchem", "2519", "MolecularFormula"]
# => {"value": "C8H10N4O2"}
```

Coverage: 100M+ compounds
Properties: MolecularFormula, MolecularWeight, IUPACName, CanonicalSMILES

2.3.2 Crossref (Scholarly Papers)

```
PackSearch["crossref", "machine learning", limit=5]
# => [{"id": "10.1145/...", "label": "Paper Title",
#      "license": "Metadata: CC0"}]

PackValue["crossref", "10.1145/3422622", "Authors"]
# => {"value": ["Author 1", "Author 2"]}
```

Coverage: 70M+ papers
Properties: Title, Authors, PublicationDate, Abstract, CitationCount

2.3.3 OpenAlex (Scholarly Graph)

```
PackSearch["openalex", "deep learning"]
# => [{"id": "W2741809807", "license": "CC0"}]

PackValue["openalex", "W2741809807", "CitationCount"]
# => {"value": 1523}
```

Coverage: 200M+ works
Properties: Title, Authors, CitationCount, Concepts, OpenAccessURL

2.3.4 GeoNames (Geographic Data)

```
PackSearch["geonames", "London"]
# => [{"id": "2643743", "license": "CC BY 4.0"}]

PackValue["geonames", "2643743", "Population"]
# => {"value": 8961989}
```

Coverage: 11M+ places
Properties: Country, Population, Latitude, Longitude, Elevation, Timezone

2.3.5 World Bank (Economic Indicators)

```
PackSearch["worldbank", "GDP"]
# => [{"id": "NY.GDP.MKTP.CD", "license": "CC BY 4.0"}]

PackValue["worldbank", "NY.GDP.MKTP.CD", "Value", country="US"]
# => {"value": 25462700000000, "date": "2022"}
```

Coverage: 1,400+ indicators
Properties: Value, Date, Country, Unit

2.3.6 AlphaFold (Protein Structures)

```
PackSearch["alphafold", "hemoglobin"]
# => [{"id": "P69905", "label": "Hemoglobin subunit alpha",
#      "license": "CC BY 4.0"}]

PackValue["alphafold", "P69905", "Confidence"]
# => {"value": 93.2}

PackValue["alphafold", "P69905", "Structure"]
# => {"value": "https://alphafold.ebi.ac.uk/files/AF-P69905-F1..."}
```

Coverage: 200M+ protein structures
Properties: Structure, Confidence, Sequence, Organism, Gene, Function

2.3.7 JHTDB (Turbulence Simulations)

```
PackSearch["jhtdb", "channel"]
# => [{"id": "channel5200", "label": "Channel Flow (Re=5200)",
#      "license": "Free for research/education"}]

PackValue["jhtdb", "channel5200", "ReynoldsNumber"]
# => {"value": 5200}

PackValue["jhtdb", "isotropic1024coarse", "Resolution"]
# => {"value": "1024^3"}
```

Coverage: 6 DNS datasets
Properties: Name, Resolution, ReynoldsNumber, TimeRange, Fields
Datasets: Isotropic turbulence, channel flow (Re=1000, Re=5200), mixing layer, MHD turbulence, transitional boundary layer

2.4 Caching & Provenance

All knowledge queries are cached in `~/.mikoshilang/knowledge_cache/knowledge.db` (SQLite) with:

- **TTL:** 7 days (Wikidata), 24 hours (Wikipedia)
- **Provenance:** Source URL, retrieved timestamp, license
- **Performance:** 200x speedup on cached queries

```
CacheStats[]
# => {"total_entries": 1523, "size_mb": 8.4}

CacheClear[30]  # Clear entries older than 30 days
```

2.5 License Compliance

Every result includes license metadata:

- **Wikidata:** CC0 (public domain)
- **Wikipedia:** CC BY-SA 3.0 (attribution required)
- **PubChem:** Public Domain
- **Crossref:** Metadata CC0
- **OpenAlex:** CC0
- **GeoNames:** CC BY 4.0
- **World Bank:** CC BY 4.0

3 Syntax

MikoshiLang uses Wolfram-inspired syntax with square brackets for function calls and curly braces for lists.

3.1 Basic Expressions

```
# Arithmetic
2 + 3 * 4       # => 14
2^10            # => 1024
Sqrt[16]        # => 4

# Variables
x + 2*y - z     # => x + 2*y - z

# Lists
{1, 2, 3, 4}    # => [1, 2, 3, 4]
Range[1, 10]    # => [1, 2, 3, ..., 10]
```

3.2 Function Calls

```
# Single argument
Sin[Pi/2]       # => 1
Exp[1]          # => E

# Multiple arguments
GCD[48, 18]     # => 6
Mod[17, 5]      # => 2

# Nested calls
Sqrt[Abs[-16]]  # => 4
```

3.3 Pattern Matching

```
# Define a rule
rule = x^2 + 2*x + 1 -> (x + 1)^2

# Apply rule
ReplaceAll[x^2 + 2*x + 1, rule]
# => (x + 1)^2
```

4 Core Functions

4.1 Calculus

4.1.1 Differentiation

```
D[x^3, x]                    # First derivative
D[x^3 + 2*x^2, x, 2]         # Second derivative
```

Available: `D`, `Derivative`, `PartialD`, `Gradient`, `Divergence`, `Curl`, `Laplacian`

4.1.2 Integration

```
Integrate[x^2, x]            # Indefinite integral
Integrate[x^2, {x, 0, 1}]    # Definite integral (0 to 1)
```

Available: `Integrate`, `NIntegrate`, `LineIntegral`, `DoubleIntegral`, `TripleIntegral`

4.1.3 Limits

```
Limit[sin(x)/x, x, 0]        # => 1
Limit[1/x, x, Infinity]      # => 0
```

Available: `Limit`, `Series`, `TaylorSeries`, `LaurentSeries`

4.2 Algebra

4.2.1 Solving Equations

```
Solve[x^2 - 4 == 0, x]       # => {-2, 2}
Solve[x^2 + x + 1 == 0, x]   # Complex solutions
NSolve[cos(x) == x, x]       # Numerical solver
```

Available: `Solve`, `NSolve`, `DSolve`, `RSolve`, `Roots`, `FindRoot`

4.2.2 Simplification

```
Simplify[(x^2 - 1)/(x - 1)]  # => x + 1
Expand[(x + 1)^3]            # => x^3 + 3*x^2 + 3*x + 1
Factor[x^2 - 4]              # => (x - 2)*(x + 2)
```

Available: `Simplify`, `FullSimplify`, `Expand`, `Factor`, `Apart`, `Together`, `Cancel`, `Collect`

4.3 Linear Algebra

```
# Determinant
Det[{{1, 2}, {3, 4}}]        # => -2

# Eigenvalues
Eigenvalues[{{1, 2}, {2, 1}}]  # => [-1, 3]

# Matrix multiplication
Dot[{{1, 2}, {3, 4}}, {{5}, {6}}]  # => {{17}, {39}}
```

Available: `Det`, `Inverse`, `Transpose`, `Eigenvalues`, `Eigenvectors`, `Dot`, `Cross`, `MatrixPower`, `MatrixRank`, `Trace`, `Norm`

4.4 Number Theory

```
Prime[100]                   # 100th prime number
PrimeQ[17]                   # True (17 is prime)
FactorInteger[60]            # => {{2, 2}, {3, 1}, {5, 1}}
GCD[48, 18]                  # => 6
LCM[12, 18]                  # => 36
```

Available: `Prime`, `PrimeQ`, `NextPrime`, `PrimePi`, `FactorInteger`, `Divisors`, `GCD`, `LCM`, `EulerPhi`, `MoebiusMu`, `Fibonacci`, `Lucas`

4.5 Statistics

```
Mean[{1, 2, 3, 4, 5}]        # => 3
Median[{1, 2, 3, 4, 5}]      # => 3
StandardDeviation[{1, 2, 3}] # => sqrt(2/3)
Variance[{1, 2, 3, 4}]       # => 5/4
```

Available: `Mean`, `Median`, `Mode`, `Variance`, `StandardDeviation`, `Quantile`, `Quartiles`, `Correlation`, `Covariance`, `Skewness`, `Kurtosis`

5 Visualization

MikoshiLang supports 2D and 3D plotting with matplotlib (static) or plotly (interactive).

5.1 2D Plotting

```
# Simple plot
Plot[sin(x), {x, 0, 6.28}]

# Multiple functions
Plot[{sin(x), cos(x)}, {x, 0, 6.28}]

# Parametric plot
ParametricPlot[{cos(t), sin(t)}, {t, 0, 6.28}]

```

```
# Polar plot
PolarPlot[1 + cos(theta), {theta, 0, 6.28}]
```

5.2 3D Plotting

```
# 3D surface
Plot3D[sin(x)*cos(y), {x, -3, 3}, {y, -3, 3}]

# Contour plot
ContourPlot[x^2 + y^2, {x, -2, 2}, {y, -2, 2}]

# Vector field
VectorFieldPlot[{-y, x}, {x, -2, 2}, {y, -2, 2}]
```

5.3 Interactive Plots

```
# Use plotly for interactive 3D
Plot3D[x^2 + y^2, {x, -2, 2}, {y, -2, 2}, interactive=True]
```

6 Chemistry

6.1 Equation Balancing

```
BalanceEquation["H2 + O2 -> H2O"]
# => 2H2 + O2 -> 2H2O

BalanceEquation["C6H12O6 -> CO2 + H2O"]
# => C6H12O6 -> 6CO2 + 6H2O
```

6.2 Molecular Mass

```
MolecularMass["H2O"]          # => 18.015 g/mol
MolecularMass["C6H12O6"]      # => 180.156 g/mol (glucose)
MolecularMass["NaCl"]         # => 58.443 g/mol
```

6.3 Element Data

```
Element["Fe"]
# => {
#   name: Iron,
#   number: 26,
#   mass: 55.845,
#   electronConfiguration: [Ar] 3d6 4s2,
#   category: transition metal
# }
```

7 Unit Conversion

MikoshiLang supports 50+ physical units across length, mass, time, temperature, energy, power, pressure, and more.

```
Convert[100, "miles", "km"]      # => 160.934 km
Convert[32, "fahrenheit", "celsius"]  # => 0 C
Convert[1, "atm", "pascal"]      # => 101325 Pa
Convert[1, "kWh", "joules"]      # => 3.6e6 J
```

Available units: meters, km, miles, feet, inches, kg, pounds, ounces, seconds, hours, days, celsius, fahrenheit, kelvin, joules, calories, kWh, watts, horsepower, atm, bar, pascal, liters, gallons

8 Natural Language Mode

MikoshiLang includes an AI-powered natural language interface. Type plain English, get verified results.

8.1 How It Works

1. User types natural language query
2. AI translates to MikoshiLang code
3. Code is executed by the deterministic engine
4. Result is verified and returned

8.2 Examples

```
# Natural language input
"What is the derivative of x cubed?"
# AI translates to: D[x^3, x]
# Engine returns: 3*x^2

"Solve x squared equals 4"
# AI translates to: Solve[x^2 == 4, x]
# Engine returns: {-2, 2}

"What is the molecular mass of water?"
# AI translates to: MolecularMass["H2O"]
# Engine returns: 18.015 g/mol
```

8.3 Web Console

The hosted web console described in earlier editions is no longer online. Use the `mikoshilang` REPL, the Jupyter kernel, or the Python API instead.

9 API Usage

9.1 Python Library

```
from mikoshilang import parse_and_eval, evaluate
from mikoshilang.parser import parse

# Parse and evaluate
result = parse_and_eval("D[x^2, x]")
print(result)  # => 2*x

# Parse only (returns AST)
ast = parse("x^2 + 2*x + 1")
```

```
print(ast)  # => BinOp(...)

# Evaluate parsed AST
result = evaluate(ast)
print(result)  # => x^2 + 2*x + 1
```

9.2 REST API

The hosted REST API described in earlier editions is no longer online and is not part of the Python package. To serve MikoshiLang over HTTP, wrap `Session.eval` in your own web service, and run untrusted input in an external process sandbox with time and memory limits.

9.3 Jupyter Kernel

MikoshiLang provides a custom Jupyter kernel for interactive Wolfram-style notebooks with LaTeX rendering and inline plots.

9.3.1 Installation

```
# Install with Jupyter support
pip install mikoshilang[jupyter]

# Install the kernel
python -m mikoshilang.jupyter.install

# Start Jupyter
jupyter notebook
```

9.3.2 Platform Support

- **Linux:** Fully supported
- **macOS:** Fully supported (Intel & Apple Silicon)
- **Windows (Anaconda):** Fully supported (fixed in v3.6.9)
- **Windows (WSL):** Fully supported

Anaconda Compatibility: As of v3.6.9, the MikoshiLang kernel is fully compatible with Anaconda on all platforms. The kernel applies `nest_asyncio.apply()` before launch to fix Windows ProactorEventLoop conflicts.

9.3.3 Usage

1. Create a new notebook in Jupyter
2. Select **MikoshiLang** from the kernel dropdown
3. Use Wolfram-style syntax directly:

```
In[1]:= x^2 + 2*x + 1
Out[1]= x^2 + 2*x + 1  (rendered as LaTeX)

In[2]:= Solve[x^2 - 4 == 0, x]
Out[2]= [-2, 2]

In[3]:= Plot2D[Sin[x], {x, -Pi, Pi}]
Out[3]= <inline plot>
```

9.3.4 Troubleshooting

If the kernel doesn't appear in Jupyter:

```
# Verify kernel is installed
jupyter kernelspec list

# Reinstall kernel
python -m mikoshilang.jupyter.install --user

# Check Python environment matches Jupyter
which python
which jupyter
```

10 Function Reference

MikoshiLang implements **4,402 of the 6,716** documented Wolfram Language symbols (see `docs/wolfram_coverage_roadmap.md` for the authoritative, generated list). The categorized listing below dates from earlier releases; entries from quarantined modules are not available in 3.8.

10.1 Version History

- **v3.8.0** (Current) — Correctness repair (generated modules quarantined); 4,402 documented Wolfram symbols with reviewed semantics; `Verify[]` and executable `Check:` notes; Python interop; opt-in types. See `CHANGELOG.md`
- **v3.3.0** — Full knowledge layer: 5 domain packs (Crossref, OpenAlex, GeoNames, World Bank, PubChem), SPARQL graph queries, persistent cache, 50 canonical properties
- **v3.2.0** — MVP knowledge layer: EntitySearch, EntityValue, WikipediaSummary, Wikidata/Wikipedia integration
- **v3.1.1** — claimed 6,324 computational functions (claim retracted in 3.8)
- **v3.1.0** — Added 600 functions: hydrology, mining, manufacturing, renewable energy
- **v3.0.4** — Fixed all placeholders, 100% working functions
- **v3.0.0** — Added 600 functions: info theory, acoustics, GIS, chemical engineering
- **v2.5.0** — Added 600 functions: materials, optics, aerospace, nuclear physics
- **v2.4.0** — Added 600 functions: climate, medical, robotics, environmental
- **v2.3.0** — Added 900 functions: engineering, biology, finance, fluids
- **v2.2.0** — Added 900 functions: CS algorithms, astronomy, statistics, NLP
- **v2.0.0** — Added 452 functions: vector calculus, ML, physics, biology
- **v1.0.0** — Initial release: 1,342 core functions

10.2 Extended Function Sets (v2.0+)

MikoshiLang v2.0+ includes 32 extended modules covering specialized scientific domains:

- **Extended 6-9** (552) — Vector calculus, ML/time series, physics, biology/numerical methods
- **Extended 10-12** (600) — Special functions, audio/signal processing, graph theory/quantum computing
- **Extended 13-16** (900) — Computer science algorithms, astronomy/astrophysics, advanced statistics, NLP
- **Extended 17-20** (900) — Mechanical/electrical/civil engineering, biology/genetics, finance/economics, fluid dynamics
- **Extended 21-24** (600) — Climate science, medical sciences, robotics, environmental science
- **Extended 25-28** (600) — Materials science, optics/photonics, aerospace engineering, nuclear/particle physics
- **Extended 29-32** (600) — Information theory/cryptography, acoustics/audio, geospatial/GIS, chemical engineering
- **Extended 33** (100) — Agriculture, food science, sports science, textile/petroleum engineering
- **Extended 34** (150) — Hydrology, groundwater, watershed analysis, oceanography, coastal engineering
- **Extended 35** (150) — Rock mechanics, mine design, drilling/blasting, mineral processing, flotation
- **Extended 36** (153) — Production planning, quality control, lean manufacturing, maintenance, ergonomics, facility layout
- **Extended 37** (150) — Solar/wind energy, energy storage, power systems, grid integration

10.3 Domain Coverage

Mathematics: Calculus, linear algebra, number theory, statistics, differential equations, discrete math, abstract algebra, topology, geometry, tensor analysis

Physics: Classical mechanics, thermodynamics, electromagnetism, quantum mechanics, relativity, astronomy, astrophysics, nuclear physics, particle physics, optics, acoustics, fluid dynamics

Engineering: Mechanical, electrical, civil, chemical, aerospace, materials, mining, manufacturing, petroleum, textile, robotics

Computer Science: Algorithms, data structures, ML, NLP, cryptography, information theory, signal processing, image processing

Natural Sciences: Chemistry, biology, genetics, environmental science, climate science, oceanography, geology, hydrology

Applied Sciences: Medicine, pharmacology, agriculture, food science, sports science, renewable energy

Industrial: Production, quality control, lean manufacturing, facility layout, maintenance, ergonomics

10.4 Calculus (67 functions)

D, Derivative, PartialD, Grad, Gradient, Div, Divergence, Curl, Laplacian, Integrate, NIntegrate, Limit, Series, TaylorSeries, LaurentSeries, FourierSeries, FourierTransform, InverseFourierTransform, LaplaceTransform, InverseLaplaceTransform, ZTransform, InverseZTransform, DSolve, NDSolve, Residue, LineIntegral, SurfaceIntegral, VolumeIntegral, DoubleIntegral, TripleIntegral, ...

10.5 Algebra (89 functions)

Solve, NSolve, DSolve, RSolve, Roots, FindRoot, Simplify, FullSimplify, Expand, Factor, Apart, Together, Cancel, Collect, Coefficient, CoefficientList, PolynomialQ, Degree, Exponent, ReplaceAll, ReplaceRepeated, Rule, Pattern, Blank, BlankSequence, ...

10.6 Linear Algebra (76 functions)

Det, Determinant, Inverse, Transpose, Eigenvalues, Eigenvectors, CharacteristicPolynomial, MinimalPolynomial, Dot, Cross, Norm, Normalize, MatrixPower, MatrixExp, MatrixLog, MatrixRank, Trace, DiagonalMatrix, IdentityMatrix, ZeroMatrix, RandomMatrix, HilbertMatrix, VandermondeMatrix, JordanForm, SchurDecomposition, QRDecomposition, LUDecomposition, CholeskyDecomposition, SVD, PseudoInverse, ...

10.7 Number Theory (94 functions)

Prime, PrimeQ, IsPrime, NextPrime, PreviousPrime, PrimePi, PrimeFactors, FactorInteger, Divisors, DivisorSigma, EulerPhi, MoebiusMu, LiouvilleLambda, Fibonacci, Lucas, Binomial, Multinomial, Factorial, DoubleFactorial, Subfactorial, GCD, LCM, ExtendedGCD, ChineseRemainder, Mod, PowerMod, JacobiSymbol, LegendreSymbol, KroneckerSymbol, ...

10.8 Statistics (112 functions)

Mean, Median, Mode, Variance, StandardDeviation, Quantile, Quartiles, InterquartileRange, Range, Correlation, Covariance, Skewness, Kurtosis, GeometricMean, HarmonicMean, TrimmedMean, CentralMoment, Moment, RandomVariate, PDF, CDF, InverseCDF, NormalDistribution, UniformDistribution, ExponentialDistribution, PoissonDistribution, BinomialDistribution, ChiSquareDistribution, StudentTDistribution, FDistribution, ...

10.9 Trigonometry (48 functions)

Sin, Cos, Tan, Cot, Sec, Csc, ArcSin, ArcCos, ArcTan, ArcCot, ArcSec, ArcCsc, Sinh, Cosh, Tanh, Coth, Sech, Csch, ArcSinh, ArcCosh, ArcTanh, ArcCoth, ArcSech, ArcCsch, Haversine, InverseHaversine, TrigExpand, TrigReduce, TrigSimplify, TrigToExp, ExpToTrig, ...

10.10 Special Functions (97 functions)

Gamma, LogGamma, PolyGamma, DiGamma, TriGamma, Beta, Zeta, HurwitzZeta, Erf, Erfc, Erfi, BesselJ, BesselY, BesselI, BesselK, AiryAi, AiryBi, HermiteH, LaguerreL, LegendreP, ChebyshevT, ChebyshevU, JacobiP, GegenbauerC, SphericalHarmonicY, EllipticK, EllipticE, EllipticF, EllipticPi, ...

10.11 Discrete Math (68 functions)

Permutations, Combinations, Multinomial, Subsets, Tuples, IntegerPartitions, SetPartitions, StirlingS1, StirlingS2, BellNumber, CatalanNumber, BernoulliB, EulerE, HarmonicNumber, GeneralizedHarmonicNumber, PartitionsP, PartitionsQ, ...

10.12 Logic & Sets (34 functions)

And, Or, Not, Xor, Nand, Nor, Implies, Equivalent, ForAll, Exists, Union, Intersection, Complement, SymmetricDifference, Subset, Element, Cardinality, PowerSet, CartesianProduct, ...

10.13 String Operations (52 functions)

StringLength, StringReverse, StringJoin, StringSplit, StringReplace, StringTake, StringDrop, ToUpperCase, ToLowerCase, StringContainsQ, StringStartsQ, StringEndsQ, StringCount, StringPosition, Characters, FromCharacterCode, ToCharacterCode, ...

10.14 List Manipulation (89 functions)

Length, First, Last, Rest, Most, Part, Take, Drop, Append, Prepend, Insert, Delete, Sort, Reverse, Flatten, Partition, Riffle, Join, Union, Intersection, Complement, Select, Map, Apply, Fold, FoldList, Nest, NestList, Table, Range, ...

10.15 Graph Theory (64 functions)

Graph, DirectedGraph, WeightedGraph, AdjacencyMatrix, IncidenceMatrix, EdgeList, VertexList, VertexCount, EdgeCount, VertexDegree, InDegree, OutDegree, PathGraph, CycleGraph, CompleteGraph, StarGraph, WheelGraph, GridGraph, TreeGraph, ShortestPath, GraphDistance, GraphDiameter, GraphRadius, ConnectedComponents, StronglyConnectedComponents, WeaklyConnectedComponents, ArticulationPoints, Bridges, ChromaticNumber, ChromaticPolynomial, MaximumFlow, MinimumCut, SpanningTree, MinimumSpanningTree, EulerianPath, HamiltonianPath, ...

10.16 Chemistry (118 elements + functions)

Element, MolecularMass, BalanceEquation, AtomicNumber, AtomicMass, ElectronConfiguration, Electronegativity, MeltingPoint, BoilingPoint, Density, H, He, Li, Be, B, C, N, O, F, Ne, Na, Mg, Al, Si, P, S, Cl, Ar, K, Ca, Fe, Cu, Zn, Ag, Au, U, ...

10.17 Units (50+ units)

Convert, Meter, Kilometer, Mile, Foot, Inch, Kilogram, Pound, Ounce, Second, Hour, Day, Celsius, Fahrenheit, Kelvin, Joule, Calorie, KilowattHour, Watt, Horsepower, Atmosphere, Bar, Pascal, Liter, Gallon, ...

10.18 Visualization (9 functions)

Plot, Plot3D, ParametricPlot, PolarPlot, ContourPlot, VectorFieldPlot, Heatmap, AnimatePlot, Interactive3D

10.19 Numerical Methods (43 functions)

`NIntegrate, NSolve, FindRoot, FindMinimum, FindMaximum, NMinimize, NMaximize, NDSolve, InterpolatingFunction, Interpolation, LinearInterpolation, CubicSplineInterpolation, BSplineInterpolation, Fit, LinearFit, PolynomialFit, NonlinearModelFit, ...`

10.20 Optimization (37 functions)

`Minimize, Maximize, NMinimize, NMaximize, LinearProgramming, ConvexOptimization, ConstrainedMin, ConstrainedMax, GradientDescent, NewtonMethod, GoldenSectionSearch, ...`

10.21 Control Theory (28 functions)

`TransferFunction, StateSpaceModel, Poles, Zeros, SystemGain, BodePlot, NyquistPlot, RootLocus, StabilityMargin, Controllability, Observability, LQR, PIDController, ...`

10.22 Signal Processing (46 functions)

`FFT, InverseFFT, DFT, IDFT, DCT, IDCT, Spectrogram, Periodogram, Correlate, Convolve, ButterworthFilter, ChebyshevFilter, BesselFilter, EllipticFilter, ...`

10.23 Time & Date (24 functions)

`Now, Today, DateString, DateList, DayName, MonthName, DayOfWeek, DayOfYear, LeapYearQ, DateDifference, DatePlus, TimeZone, UnixTime, ...`

10.24 Random & Probability (38 functions)

`Random, RandomInteger, RandomReal, RandomChoice, RandomSample, SeedRandom, RandomVariate, RandomPrime, RandomGraph, RandomMatrix, ...`

10.25 Financial Math (21 functions)

`PresentValue, FutureValue, NPV, IRR, Annuity, TimeValue, CompoundInterest, ContinuouslyCompo BondPrice, YieldToMaturity, ...`

11 New Domain Examples (v2.0)

11.1 Vector Calculus

```
# Gradient of a scalar field
Grad[x^2 + y^2 + z^2, {x, y, z}]
# => [2*x, 2*y, 2*z]

# Divergence of a vector field
Div[{x, y, z}, {x, y, z}]
# => 3

# Curl of a vector field
Curl[{-y, x, 0}, {x, y, z}]
# => [0, 0, 2]

# Laplacian
Laplacian[x^2 + y^2, {x, y}]
```

```
# => 4
```

11.2 Machine Learning

```
# Linear regression
X = Matrix([[1, 2], [2, 3], [3, 4]])
y = Matrix([3, 5, 7])
LinearRegression[X, y]

# Activation functions
Sigmoid[x]        # => 1/(1 + exp(-x))
ReLU[x]           # => Max(0, x)
Softmax[{x, y, z}]

# Loss functions
MeanSquaredError[ytrue, ypred]
CrossEntropy[p, q]

# Metrics
Accuracy[ytrue, ypred]
F1Score[precision, recall]
```

11.3 Physics

```
# Classical mechanics
KineticEnergy[m, v]           # => m*v^2/2
PotentialEnergy[m, g, h]      # => m*g*h
EscapeVelocity[M, r]          # => sqrt(2*G*M/r)

# Thermodynamics
IdealGasLaw[P, V, n, T]       # => P*V - n*R*T
CarnotEfficiency[Th, Tc]      # => 1 - Tc/Th

# Electromagnetism
CoulombForce[q1, q2, r]       # => k_e*q1*q2/r^2
OhmLaw[V, R]                  # => V/R

# Quantum mechanics
PlankEnergy[freq]             # => h*freq
DeBroglieWavelength[p]        # => h/p
HeisenbergUncertainty[dx, dp] # => dx*dp >= hbar/2

# Relativity
LorentzFactor[v]              # => 1/sqrt(1 - v^2/c^2)
MassEnergyEquivalence[m]      # => m*c^2
```

11.4 Biology & Genetics

```
# DNA analysis
Complement["A"]               # => "T"
ReverseComplement["ATCG"]     # => "CGAT"
Transcribe["ATCG"]            # => "AUCG"
GCContent["ATCGGC"]           # => 0.5

# Molecular mass
MolecularMass["H2O"]          # => 18.015 g/mol

# Population dynamics
LogisticGrowth[N, r, K]       # => r*N*(1 - N/K)
```

```
SIRModel[S, I, R, beta, gamma]

# Sequence alignment
HammingDistanceDNA[seq1, seq2]
LevenshteinDNA[seq1, seq2]
```

11.5 Cryptography

```
# Hashing
SHA256["hello"]
SHA512["world"]
MD5["message"]

# Modular arithmetic
ModularExponentiation[base, exp, mod]
ModularInverse[a, m]
ExtendedEuclidean[a, b]

# Number theory
DiscreteLog[base, result, modulus]
PrimitiveRoot[p]
QuadraticResidue[a, p]
LegendreSymbol[a, p]
```

11.6 Numerical Methods

```
# Root finding
NewtonRaphson[f, x0, tol]
BisectionMethod[f, a, b, tol]
SecantMethod[f, x0, x1, tol]

# Integration
TrapezoidalRule[f, a, b, n]
SimpsonsRule[f, a, b, n]
GaussianQuadrature[f, a, b, n]

# ODEs
EulerMethod[f, y0, t0, h, n]
RungeKutta4[f, y0, t0, h]
AdamsBashforth[f, y, h]

# Interpolation
LagrangeInterpolation[{x1, x2, x3}, {y1, y2, y3}]
CubicSpline[points]
```

12 Advanced Topics

12.1 Custom Rules

Define your own transformation rules:

```
from mikoshilang import parse_and_eval

# Define a custom simplification rule
rule = "sin(x)^2 + cos(x)^2 -> 1"

# Apply it
expr = "sin(x)^2 + cos(x)^2 + 2*x"
result = parse_and_eval(f"ReplaceAll[{expr}, {rule}]")
# => 1 + 2*x
```

12.2 Pattern Matching

```
# Match any power
Pattern[_, x^_]

# Match specific forms
Pattern[a_ + b_, ...]
```

12.3 Performance Tips

- Use `NSolve` for numerical roots instead of symbolic `Solve`
- Cache repeated computations
- Use `N[expr]` to convert symbolic to numeric
- Simplify expressions before integration/differentiation
- For large matrices, use sparse representations

13 Troubleshooting

13.1 Common Errors

SyntaxError: Invalid expression
Ensure square brackets for function calls and proper operator syntax.
NameError: Function not found
Check spelling and capitalization. MikoshiLang functions are case-sensitive (`Sin` not `sin`).
ValueError: Cannot parse expression
Verify parentheses/brackets are balanced.

13.2 Getting Help

- GitHub Issues: https://github.com/DarrenEdwards111/MikoshiLang/issues
- Documentation: https://github.com/DarrenEdwards111/MikoshiLang/blob/main/README.md

14 Contributing

MikoshiLang is open-source (Apache 2.0). Contributions welcome:

1. Fork the repository
2. Create a feature branch
3. Add or fix functions in `mikoshilang/library/`, following `docs/CONVENTIONS.md`, with an executable `Check:` note
4. Write tests in `tests/`
5. Submit a pull request

15 License

MikoshiLang is licensed under the Apache License 2.0.

```
Copyright 2025 Mikoshi Ltd

Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0
```

16 Acknowledgments

MikoshiLang is built on:

- **SymPy** — symbolic mathematics
- **matplotlib** — 2D plotting
- **plotly** — interactive visualization
- **numpy** — numerical arrays

Inspired by Wolfram Language and designed for the Python ecosystem.

Built by Mikoshi Ltd
https://github.com/DarrenEdwards111/MikoshiLang